



Technical Report – H-Bridge with Relays

T3 - Specialist Technician in Automation,
Robotics, and Industrial Control

Bruna Ferreira Gomes Madeira



Circuit Parameters

The circuit uses an NPN transistor (2N3904) to drive a relay coil.

Given parameters:

Supply voltage:

$$V_{in} = 5V$$

Base-emitter voltage (from datasheet):

$$V_{be} = 0,65V \text{ to } 0.85V$$

Typical value used for calculations:

$$V_{be} \approx 7V$$

Relay coil current (collector current):

$$I_C = 40mA$$

Forced current gain for saturation:

$$\beta = 10$$

Base Current Calculation

The required base current is determined by:

$$\beta = \frac{I_C}{I_\beta}$$

$$I_\beta = \frac{I_C}{\beta}$$

$$I_\beta = \frac{40}{10}$$

$$I_\beta = 0,004A$$

Base Resistor Calculation

The base resistor limits the base current according to:

$$R_\beta = \frac{V_{in} - V_{be}}{I_\beta}$$



$$R_{\beta} = \frac{5 - 0,7}{0,004}$$

$$R_{\beta} = \frac{4,3}{0,004}$$

$$R_{\beta} = 1075\Omega$$

Selection of a Standard Resistor Value

The closest standard resistor value is:

$$R_{\beta} = 1k\Omega$$

A 1 kΩ base resistor provides sufficient base current to drive the 2N3904 transistor into saturation, ensuring reliable switching of the relay coil.

Relay Shield Simulation – Tinkercad

The relay shield was modeled and simulated in [Tinkercad](#) to validate its operation in controlling multiple loads via a microcontroller. The simulation setup included:

- Connection of the relay module to the microcontroller's digital output pins.
- Activation and deactivation of individual relays to confirm switching functionality.
- Observation of integrated LED indicators as feedback for relay status.
- Emulated load testing to verify proper relay response under operational conditions.

The simulation demonstrated that the relay shield provides reliable electrical isolation and control of external circuits, ensuring safe operation of high-current devices without direct exposure to the microcontroller.

H-Bridge Simulation – Wokwi

The H-bridge circuit was simulated in [Wokwi](#) to assess its capability for bidirectional control of a DC motor. The simulation encompassed:

- Connections between the H-bridge inputs and the microcontroller's output pins.
- Logic-level manipulation to control motor rotation direction.



- Real-time visualization of motor response to directional commands.

The simulation confirmed that the H-bridge can efficiently manage motor direction, offering robust and safe operation across varying load conditions.

Base System Code

```
#include <Arduino.h>
#include <LiquidCrystal_I2C.h>

// ----- LCD -----
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ----- BOTÕES -----
#define BTN_CW 7
#define BTN_CCW 8
#define BTN_PRESSED LOW

// ----- SAÍDAS MOTOR -----
#define MOTOR_CW 3
#define MOTOR_CCW 4

#define MOTOR_ON HIGH
#define MOTOR_OFF LOW

// ----- ESTADO -----
enum MotorState { STOPPED, CW, CCW };
MotorState currentState = STOPPED;

// ----- FUNÇÕES -----

void stopMotor() {
    digitalWrite(MOTOR_CW, MOTOR_OFF);
    digitalWrite(MOTOR_CCW, MOTOR_OFF);
}

void driveCW() {
    digitalWrite(MOTOR_CCW, MOTOR_OFF);
    delay(10);
    digitalWrite(MOTOR_CW, MOTOR_ON);
}

void driveCCW() {
    digitalWrite(MOTOR_CW, MOTOR_OFF);
    delay(10);
```



```
digitalWrite(MOTOR_CCW, MOTOR_ON);
}

// ----- LCD -----

void showPresentation() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Ponte H");
  lcd.setCursor(0, 1);
  lcd.print("Autores: Brunos");
  delay(3000);
}

void updateDisplay(MotorState state) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Sistema Pronto");

  lcd.setCursor(0, 1);

  if (state == STOPPED)
    lcd.print("Motor: PARADO ");
  else if (state == CW)
    lcd.print("Motor: HORARIO ");
  else
    lcd.print("Motor: ANTI-HOR ");
}

// ----- SETUP -----

void setup() {

  pinMode(BTN_CW, INPUT_PULLUP);
  pinMode(BTN_CCW, INPUT_PULLUP);

  pinMode(MOTOR_CW, OUTPUT);
  pinMode(MOTOR_CCW, OUTPUT);

  stopMotor();

  lcd.init();
  lcd.backlight();

  showPresentation();
  updateDisplay(STOPPED);
}
```



// ----- LOOP -----

```
void loop() {  
  
    bool cwPressed = (digitalRead(BTN_CW) == BTN_PRESSED);  
    bool ccwPressed = (digitalRead(BTN_CCW) == BTN_PRESSED);  
  
    MotorState newState = STOPPED;  
  
    if (cwPressed && ccwPressed) {  
        stopMotor();  
        newState = STOPPED;  
    }  
    else if (cwPressed) {  
        driveCW();  
        newState = CW;  
    }  
    else if (ccwPressed) {  
        driveCCW();  
        newState = CCW;  
    }  
    else {  
        stopMotor();  
        newState = STOPPED;  
    }  
  
    if (newState != currentState) {  
        currentState = newState;  
        updateDisplay(currentState);  
    }  
  
    delay(20);  
}
```